

Rapport du projet DIT Librairie

Document modifiable Word - thème vert sombre / vert institutionnel

Projet: application de bibliothèque numérique distribuée en microservices, avec pipeline de recommandation versionné par DVC.

1. Présentation du projet

DIT Librairie est une application de bibliothèque numérique destinée à gérer un catalogue de livres, les comptes des lecteurs, les emprunts, les retours, les retards, les statistiques administratives et les recommandations personnalisées.

- Frontend HTML/CSS/JavaScript exposé sur le port 8080.
- Microservices Django REST pour les livres, les utilisateurs, les emprunts et les statistiques.
- Service FastAPI dédié aux recommandations, basé sur un modèle hybride collaboratif et sémantique.
- Base MySQL 8.4 orchestrée par Docker Compose, avec profils dev et prod.
- Pipeline DVC pour préparer les données, entraîner le modèle et publier les métriques.

2. Architecture du système

L'architecture est organisée autour d'un frontend consommant plusieurs APIs HTTP/JSON. Les services métiers conservent leur responsabilité fonctionnelle et communiquent au besoin avec les autres services via des variables d'environnement Docker.

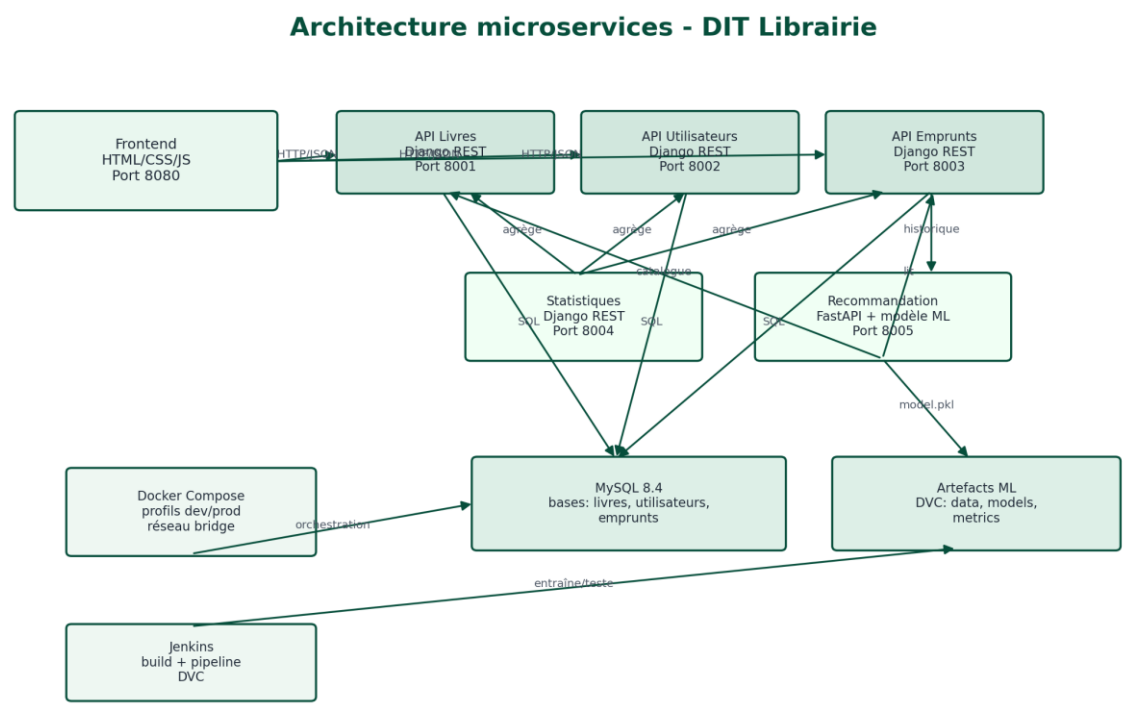


Schéma d'architecture système

3. Services

Service	Technologie	Port	Responsabilités
frontend	HTML/CSS/JavaScript	8080	Pages accueil, catalogue, connexion, inscription et compte utilisateur.
livres	Django REST	8001	Catalogue, recherche, livres disponibles/récents, stock et demandes de livres.
utilisateurs	Django REST	8002	Inscription, connexion, profils, rôles, comptes actifs et

			administration.
emprunts	Django REST	8003	Création d'emprunts, retours, historique, retards et export CSV.
statistiques	Django REST	8004	Tableau de bord admin, top livres, répartition utilisateurs, retards et santé des services.
recommandation	FastAPI	8005	Chargement du modèle, recommandations utilisateur, ré-entraînement, métriques et informations modèle.
mysql	MySQL 8.4	3306	Persistance des bases séparées livres_db, utilisateurs_db et emprunts_db.

4. APIs et échanges

Les échanges sont principalement en JSON. Le frontend ajoute des en-têtes de session simples: X-User-Id, X-User-Role et X-User-Name. Ces en-têtes pilotent les règles d'accès côté API.

Service	Endpoints principaux	Données échangées
Livres	/api/livres/, /api/livres/disponibles/, /api/livres/recents/, /api/livres/<id>/stock/, /api/demandes-livres/	Livre: titre, auteur, isbn, catégorie, description, langue, dates, image_url, quantités, statut actif. Demande: utilisateur, titre, auteur, statut, commentaire gestionnaire.
Utilisateurs	/api/utilisateurs/connexion/, /api/utilisateurs/, /api/utilisateurs/me/, /api/utilisateurs/types/, /api/utilisateurs/<id>/	Utilisateur: nom, prénom, email, téléphone, département, numéro de carte, rôle, actif. Connexion: identifiant et mot_de_passe.
Emprunts	/api/emprunts/, /api/emprunts/<id>/retour/, /api/emprunts/utilisateur/<id>/, /api/emprunts/retards/, /api/emprunts/export/csv/	Emprunt: utilisateur_id, utilisateur_nom, livre_id, livre_titre, dates, statut, commentaire. Appelle livres pour vérifier et ajuster le stock, utilisateurs pour valider l'emprunteur.
Statistiques	/api/statistiques/tableau-de-bord/, /top-livres/, /repartition-utilisateurs/, /retards/, /sante-services/	Agrège les réponses des APIs livres, utilisateurs et emprunts pour produire indicateurs et listes administratives.
Recommandation	/health, /recommendations/<user_id>, /train, /metrics, /model-info	Lit model.pkl et metrics.json, récupère le catalogue et l'historique d'emprunts, retourne une liste de recommandations notées.

5. Workflow Git utilisé

Le dépôt observé est sur la branche validation. L'historique montre une intégration de branches de contribution dans cette branche via plusieurs commits de merge, par exemple origin/fabien, origin/marieange, origin/mamadousaidoubah, origin/jurgen, origin/igor, origin/dudumf, origin/davy et origin/cheikhabdoukhadremballo.

Chaque contributeur travaille sur une branche dédiée à un service ou à une fonctionnalité.

Les branches sont fusionnées dans validation pour centraliser l'intégration.

Les conflits sont résolus sur validation avant stabilisation.

Le Jenkinsfile automatise la préparation Python, l'exécution du pipeline DVC puis le build Docker de production.

6. Pipeline DVC

Le fichier dvc.yaml définit trois étapes: preprocess, train et evaluate. Les entrées et sorties sont versionnées afin de reproduire le modèle et les métriques.

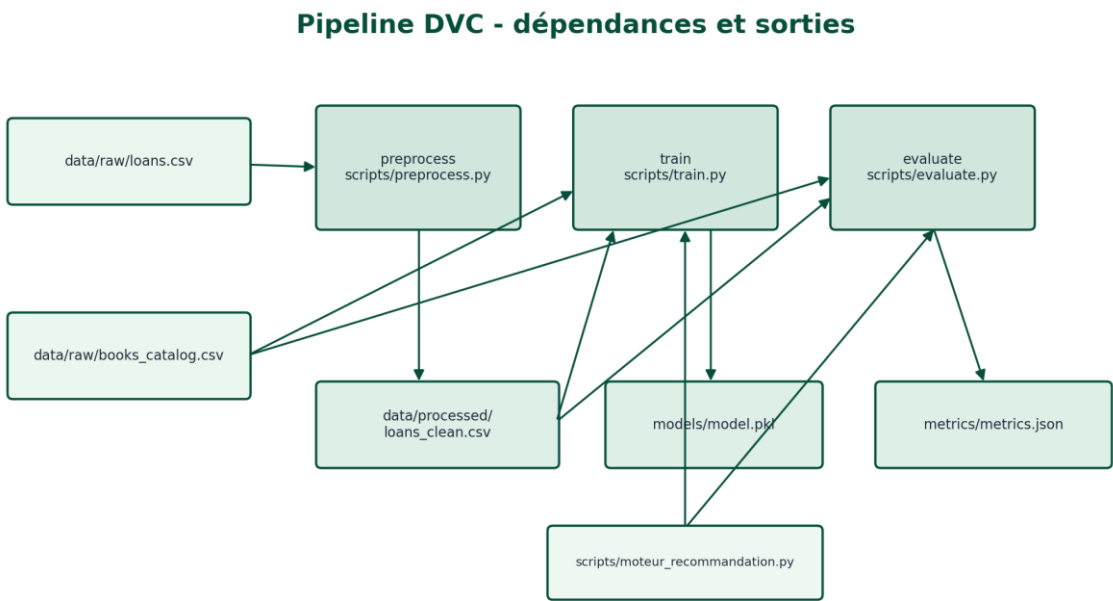


Schéma des dépendances DVC

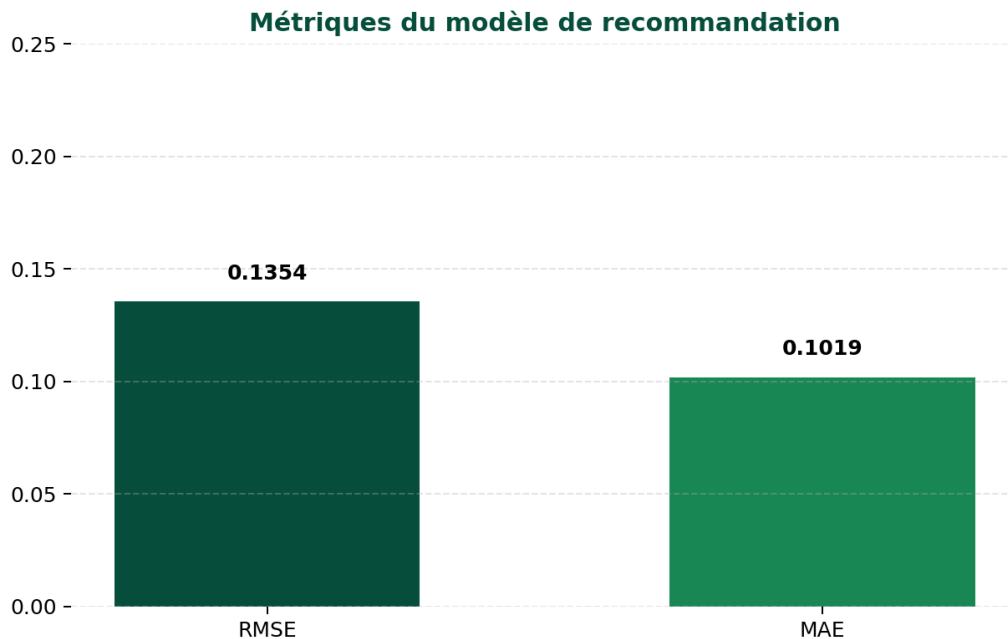
Étape	Commande	Dépendances	Sorties
preprocess	python scripts/preprocess.py	data/raw/loans.csv, scripts/preprocess.py	data/processed/loans_clean.csv
train	python scripts/train.py	loans_clean.csv, books_catalog.csv, scripts/train.py, moteur_recommandation.py	models/model.pkl
evaluate	python scripts/evaluate.py	loans_clean.csv, books_catalog.csv, scripts/evaluate.py, moteur_recommandation.py	metrics/metrics.json

7. Métriques du modèle

Algorithme: Hybride collaboratif + sémantique. Modèle sémantique: AventIQ-AI/all-MiniLM-L6-v2-book-recommendation-system.

Métrique	Valeur
RMSE	0.1354
MAE	0.1019
Observations de test	7

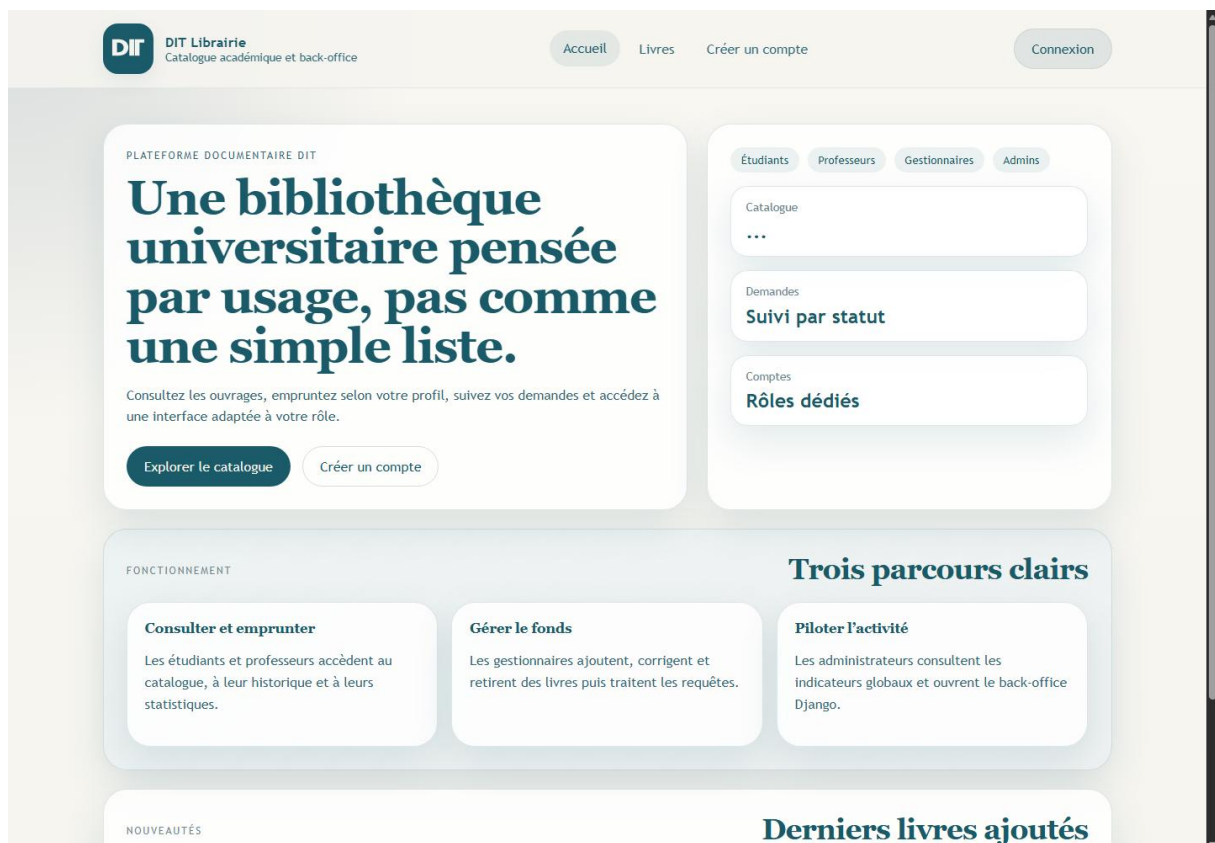
Utilisateurs train	7
Livres train	5



Visualisation des métriques RMSE et MAE

8. Captures d'écran

Les captures ci-dessous ont été générées depuis les pages frontend du projet. Si les APIs backend ne sont pas lancées au moment de la capture, certains blocs dynamiques peuvent rester vides ou afficher un état d'erreur.





DIT Librairie
Catalogue académique et back-office

[Accueil](#) [Livres](#) [Créer un compte](#)

Connexion

CATALOGUE

Trouvez un livre en quelques filtres.

Tous

▼

☐ Disponibles uniquement

Rechercher

RECHERCHE

Résultats de la recherche

SÉLECTION

Derniers livres ajoutés

Catalogue DIT

Recherche, disponibilité et recommandations.

Page catalogue et recherche de livres



DIT Librairie
Catalogue académique et back-office

[Accueil](#) [Livres](#) [Créer un compte](#)

Connexion

CONNEXION

Accéder à votre espace

Email, nom d'utilisateur ou numéro de carte

Mot de passe

Se connecter

Pas de compte ? Créer un compte

Page de connexion

9. Utilité de dvc.lock

Le fichier dvc.lock est le journal de reproductibilité du pipeline DVC. Il fige, pour chaque étape, la commande exécutée, les dépendances exactes, les sorties produites, les tailles et les empreintes md5.

Il permet à DVC de savoir si une étape doit être rejouée ou si ses sorties sont encore valides.

Il garantit qu'un autre membre de l'équipe peut reproduire le même pipeline à partir des mêmes versions de données et de scripts.

Il documente les artefacts produits: `loans_clean.csv`, `model.pkl` et `metrics.json`.

Il facilite les revues Git, car une modification du modèle ou des données devient visible par changement d'empreinte.

10. Informations utiles

Lancement dev: `docker compose --profile dev up --build`.

Lancement prod local: `docker compose --profile prod up --build`.

Compte admin par défaut: `ditadmin / AdminDIT123!`.

Compte gestionnaire par défaut: `manager1 / manager`.

Les migrations Django et les seeds d'administration sont exécutés automatiquement au démarrage des services concernés.

Le frontend utilise `localStorage` pour conserver la session et envoie le rôle dans les en-têtes HTTP.

Les services statistiques et recommandation sont des services d'agrégation: ils ne possèdent pas la donnée métier principale, ils la récupèrent depuis les APIs existantes.